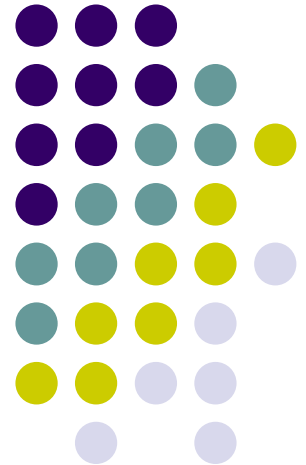
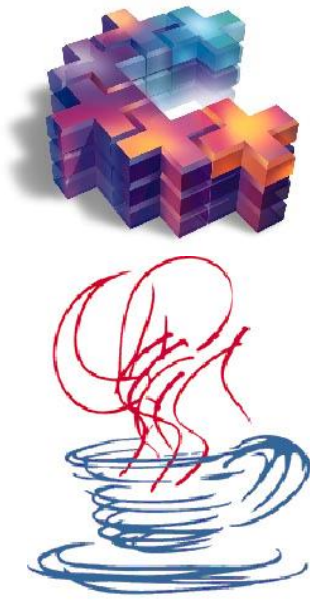


TECHNIQUES DE COMPILATION

CH2: ANALYSE SYNTAXIQUE

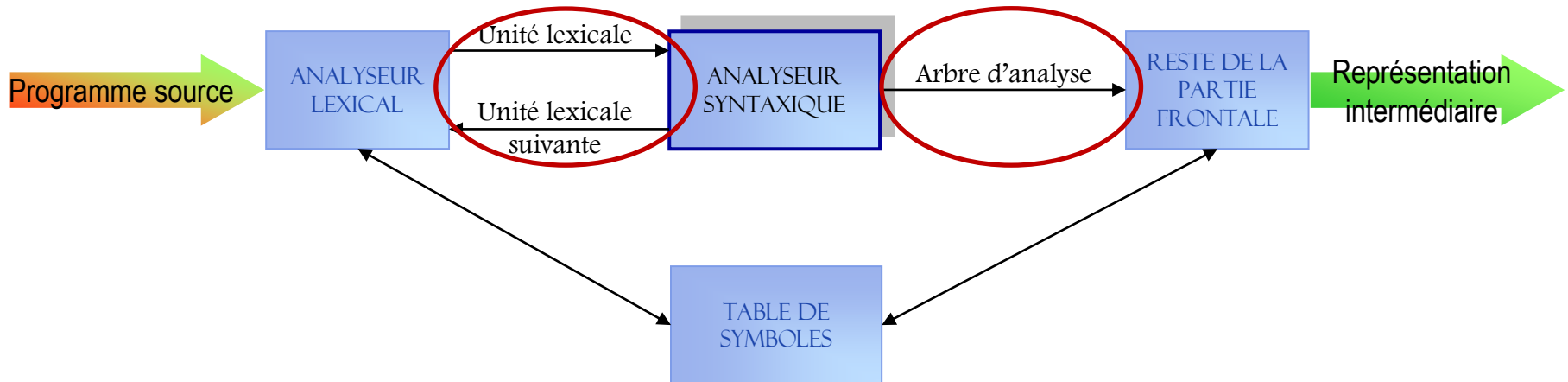


INTRODUCTION

- But de l'analyse syntaxique : transformer un flot d'unités lexicales en arbre abstrait.
- Langage de programmation : règles prescrivant la structure syntaxique des programmes bien formés.
- Langage C , Programme = ensemble de Fonctions.
 - Fonction : formée de blocs.
 - Un bloc : formé d'instructions.
 - Une instruction : formée d'expressions.
 - Une expression : formée d'unités lexicales.
 - Etc.
- La syntaxe d'un langage peut être décrite par des grammaires non contextuelles (notation BNF – Backus-Naur Form).
- Grammaire : spécification syntaxique précise et facile à comprendre d'un langage de programmation.

RÔLE DE L'ANALYSEUR SYNTAXIQUE

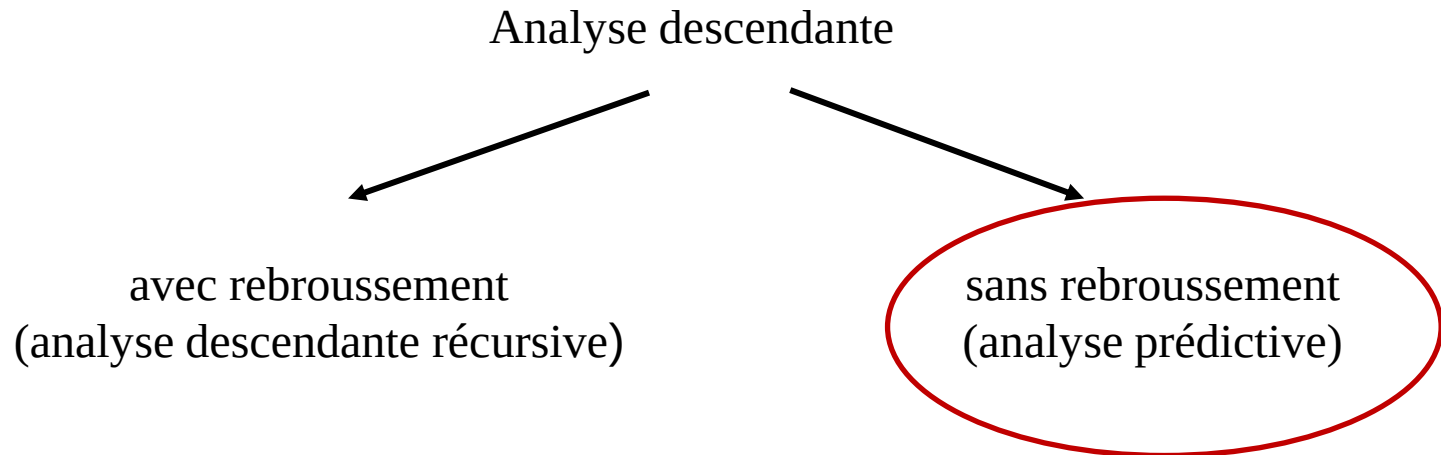
- L'analyseur syntaxique obtient une chaîne d'unités lexicales de l'analyseur lexical.
- Il vérifie que la chaîne peut être engendrée par la grammaire du langage source.
- Il signale chaque erreur de syntaxe de manière intelligible.
- Supporte les erreurs les plus communes de façon à pouvoir continuer l'analyse du code restant.



RÔLE DE L'ANALYSEUR SYNTAXIQUE

- Deux types d'analyses sont retenues pour les compilateurs : Ascendante ou Descendante.
- Analyseurs descendants construisent les arbres d'analyse de haut (ROOT) en bas (LEAF).
- Analyseurs ascendants partent des feuilles et remontent à la racine.
- Dans les deux cas l'entrée de l'analyse syntaxique est explorée par convention de la gauche vers la droite un symbole à la fois.
- Le gestionnaire d'erreurs doit :
 - Indiquer la présence d'erreurs de façon claire et précise.
 - Traiter chaque erreur pour pouvoir détecter les erreurs suivantes.
- Le gestionnaire d'erreurs ne doit pas pénaliser l'exécution des programmes correctes.

ANALYSE SYNTAXIQUE DESCENDANTE



1. Non ambiguë
2. Factorisée à gauche
3. Non récursive à gauche

FACTORISATION À GAUCHE

ALGORITHME FACTORISATION À GAUCHE

DONNÉES : Une grammaire G sans cycles et sans productions vides

RÉSULTAT : Une grammaire équivalente à G factorisée à gauche

Pour chaque non terminal A trouver le plus long préfixe α au moins à deux de ses alternatives.

Si $\alpha \neq \varepsilon$ alors remplacer toutes les A -productions :

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$$

γ Représente toutes les alternatives ne commençant pas par α par :

$$A \rightarrow \alpha A' \mid \gamma$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

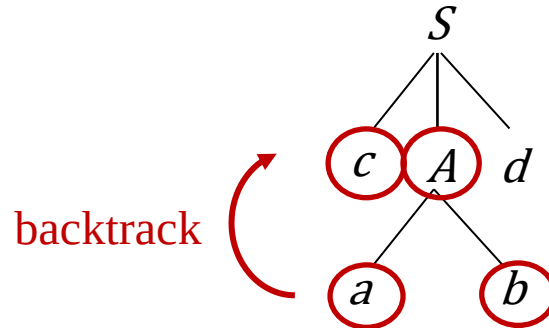
Répéter jusqu'à ce qu'aucune des alternatives d'un même non-terminal n'ait de préfixe commun.

EXEMPLE

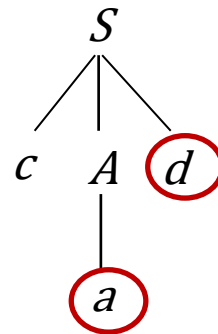
$\{ S \rightarrow cAd$
 $\{ A \rightarrow ab \mid a$
 $w = cad$

$c \underline{a} d$
 $\uparrow \uparrow \uparrow$

retour



échec
retour au choix précédent



Succès

$S \rightarrow c \underline{A} d$

$S \rightarrow c \underline{A} d \rightarrow c a b d$

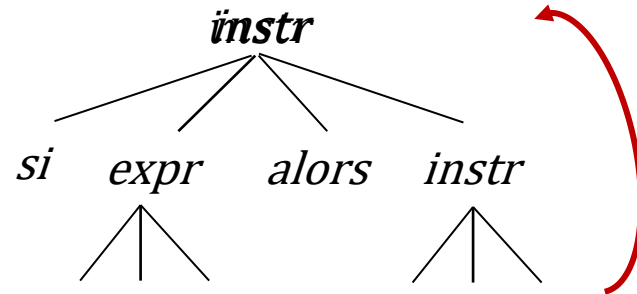
$S \rightarrow c \underline{A} d \rightarrow c a d$

Dans cet exemple, nous traitons une forme générale d'analyse descendante où des retours arrières sont possibles avec passages répétés sur le texte source. Dans la pratique, le rebroussement est rarement nécessaire, on peut adapter les grammaires pour effectuer une analyse descendante sans rebroussement

EXEMPLE

$$\left\{ \begin{array}{l} instr \rightarrow si\ expr\ alors\ instr \\ \quad | \quad si\ expr\ alors\ instr\ sinon\ instr \\ \quad | \quad autre \end{array} \right.$$

Quelle alternative choisir après lecture du si ?



backtrack

$w = si\ (vitesse \geq 110)\ alors\ amende = true\ sinon\ amende = false$



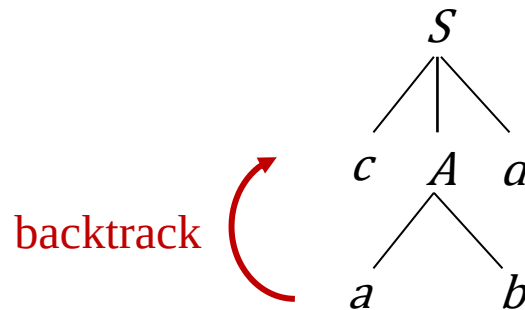
retour

EXEMPLE

Quelle est la cause du problème ?

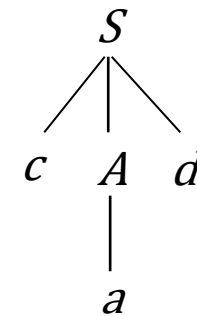
$$\begin{cases} S \rightarrow cAd \\ A \rightarrow \underline{ab} \mid \underline{a} \end{cases}$$

$w = cad$



échec
retour au choix précédent

$$\begin{cases} instr \rightarrow \underline{si\ expr\ alors\ instr} \\ \quad \quad \quad | \underline{si\ expr\ alors\ instr\ sinon\ instr} \\ \quad \quad \quad | autre \end{cases}$$



$$S \rightarrow c \underline{A} d$$

$$S \rightarrow c \underline{A} d \not\rightarrow c a b d$$

$$S \rightarrow c \underline{A} d \rightarrow c a d$$

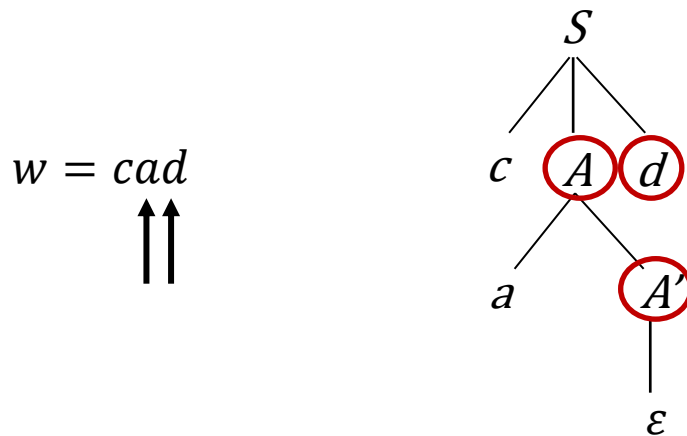
➡ Le problème c'est la factorisation à gauche

Proposer une grammaire équivalente qui dénote le même langage tout en éliminant le problème ?

FACTORISATION À GAUCHE

EXEMPLE

$$\begin{cases} S \rightarrow cAd \\ A \rightarrow \underline{ab} \mid \underline{a} \end{cases} \xrightarrow{\text{Factorisation à gauche}} \begin{cases} S \rightarrow cAd \\ A \rightarrow a(b \mid \varepsilon) \end{cases} \quad \begin{cases} S \rightarrow cAd \\ A \rightarrow aA' \\ A' \rightarrow b \mid \varepsilon \end{cases}$$



Succès

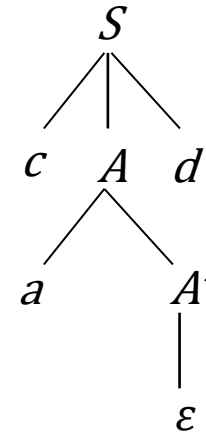
$$S \rightarrow c A d \rightarrow c a A' d \rightarrow c a d$$

Proposer une grammaire équivalente qui dénote le même langage tout en éliminant le problème ?

FACTORISATION À GAUCHE

EXEMPLE

$$\left\{ \begin{array}{l} S \rightarrow cAd \\ A \rightarrow \underline{ab} \mid \underline{a} \end{array} \right. \xrightarrow{\text{Factorisation à gauche}} \left\{ \begin{array}{l} S \rightarrow cAd \\ A \rightarrow aA' \\ A' \rightarrow \underline{b} \mid \varepsilon \end{array} \right.$$



Pour généraliser : si

$$S \rightarrow \underline{\alpha}\beta_1 \mid \underline{\alpha}\beta_2 \mid \cdots \mid \underline{\alpha}\beta_n \mid \gamma_1 \mid \cdots \mid \gamma_m \xrightarrow{\text{Factorisation à gauche}} \left\{ \begin{array}{l} S \rightarrow \underline{\alpha}S' \mid \gamma_1 \mid \cdots \mid \gamma_m \\ S' \rightarrow \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n \end{array} \right.$$

$$\left\{ \begin{array}{l} S \rightarrow aRbc \mid abc \\ R \rightarrow aRTb \mid aTb \\ T \rightarrow bb \mid cc \end{array} \right. \xrightarrow{\text{Factorisation à gauche}} \left\{ \begin{array}{l} S \rightarrow aS' \\ S' \rightarrow Rbc \mid bc \\ R \rightarrow aR' \\ R' \rightarrow RTb \mid Tb \\ T \rightarrow bb \mid cc \end{array} \right.$$

RÉCURSIVITÉ À GAUCHE ET À DROITE

- Une grammaire est réursive si elle contient un non terminal A qui se dérive en une production contenant A .
- Une production d'une grammaire est réursive à gauche si le symbole le plus à gauche de la partie droite de la production est le même que le non terminal de la partie gauche de la production (il existe une dérivation $A \Rightarrow A\alpha$ (α une chaîne quelconque)).

EXEMPLE : $A \rightarrow A\alpha$

- Une production d'une grammaire est réursive à droite si le symbole le plus à droite de la partie droite de la production est le même que le non terminal de la partie gauche de la production.

EXEMPLE : $A \rightarrow \alpha A$

α est une suite de terminaux et non terminaux ne contenant pas A .

SUPPRESSION DE LA RÉCURSIVITÉ À GAUCHE

- Les méthodes d'analyse descendante ne fonctionnent pas avec les grammaires récursives à gauche.
- L'analyseur par descente récursive peut boucler indéfiniment.
- Pour les analyseurs descendants, il faut éliminer la récursivité à gauche.
- Des règles de réécriture pour éliminer la récursivité à gauche.
- A est non terminal dans les deux productions suivantes :

$$A \rightarrow A\alpha \mid \beta$$

α et β sont des suites de terminaux et non terminaux ne contenant pas A .

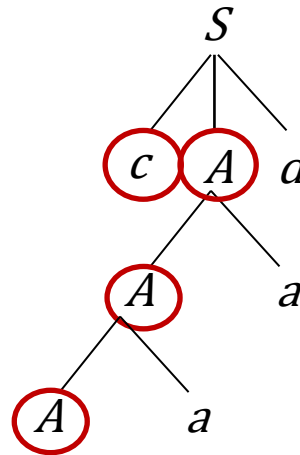
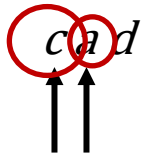
EXEMPLE : $expression \rightarrow expression + term \mid term$

RÉCURSIVITÉ À GAUCHE

EXEMPLE

$$\begin{cases} S \rightarrow cAd \\ \underline{A} \rightarrow \underline{A}a \end{cases} \mid a$$

$w = cad$



Boucle infinie

$$S \rightarrow c \underline{A} d$$

$$S \rightarrow c \underline{A} d \rightarrow c A a d$$

$$S \rightarrow c \underline{A} d \rightarrow c A a d \rightarrow c A a a d$$

...

Le problème c'est que la grammaire est **récursive à gauche**

Parfois même les retours arrières ne sont possibles !

RÉCURSIVITÉ À GAUCHE

EXEMPLE

Proposez des mots du langage ?

$$\underline{A} \rightarrow \underline{A\alpha} \mid \underline{\beta}$$

$$\beta$$

$$\beta\alpha \cdots \alpha$$

~~$$\alpha$$

$$\alpha \cdot \alpha$$

$$\alpha \cdots \alpha\beta$$~~

$$A \rightarrow A\alpha \rightarrow A\alpha\alpha \rightarrow A\alpha\alpha\alpha \rightarrow \beta\alpha\alpha\alpha$$

Proposez une grammaire qui dénote le même langage sans quelle ne soit **récursive à Gauche** ?

$$\begin{cases} A \rightarrow \underline{\beta A'} \\ A' \rightarrow \underline{\alpha A'} \mid \underline{\varepsilon} \end{cases}$$

Si on généralise :

Après élimination de la récursivité

$$\left\{ \underline{A} \rightarrow \underline{A\alpha_1} \mid \underline{A\alpha_2} \mid \cdots \mid \underline{A\alpha_n} \mid \underline{\beta_1} \mid \cdots \mid \underline{\beta_m} \right\} \longrightarrow \left\{ \begin{array}{l} A \rightarrow \underline{\beta_1 A'} \mid \cdots \mid \underline{\beta_m A'} \\ A' \rightarrow \underline{\alpha_1 A'} \mid \underline{\alpha_2 A'} \mid \cdots \mid \underline{\alpha_n A'} \mid \underline{\varepsilon} \end{array} \right.$$

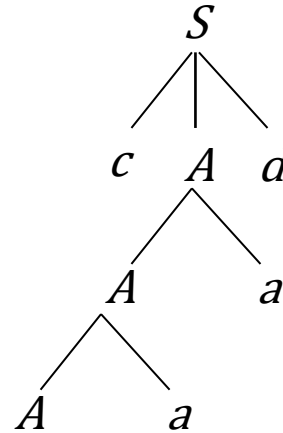
RÉCURSIVITÉ À GAUCHE

EXEMPLE

$$\begin{cases} S \rightarrow cAd \\ \underline{A} \rightarrow \underline{A}a \mid a \end{cases}$$

$w = cad$

$c a d$
↑ ↑ ↑



Boucle infinie

$$S \rightarrow c \underline{A} d$$

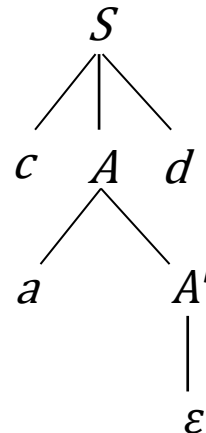
$$S \rightarrow c \underline{A} d \rightarrow c A a d$$

$$S \rightarrow c \underline{A} d \rightarrow c A a d \rightarrow c A a a d$$

...

Après élimination de la récursivité :

$$\begin{cases} S \rightarrow cAd \\ A \rightarrow aA' \\ A' \rightarrow aA' \mid \varepsilon \end{cases}$$



Succès

RÉCURSIVITÉ À GAUCHE

EXEMPLE

Si on généralise :

Après élimination de la récursivité

$$\{\underline{A} \rightarrow \underline{A}\alpha_1 \mid \underline{A}\alpha_2 \mid \cdots \mid \underline{A}\alpha_n \mid \beta_1 \mid \cdots \mid \beta_m \} \longrightarrow \begin{cases} A \rightarrow \beta_1 A' \mid \cdots \mid \beta_m A' \\ A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_n A' \mid \varepsilon \end{cases}$$

Après élimination de la récursivité

$$\begin{cases} \underline{S} \rightarrow aS \mid \underline{S}b \mid c \\ aSb \rightarrow Sa \mid bS \end{cases} \longrightarrow \begin{cases} S \rightarrow aSS' \mid cS' \\ S' \rightarrow bS' \mid \varepsilon \\ aSb \rightarrow Sa \mid bS \end{cases}$$

RÉCURSIVITÉ À GAUCHE

EXEMPLE

Si on généralise :

Après élimination de la récursivité

$$\{\underline{A} \rightarrow \underline{A}\alpha_1 \mid \underline{A}\alpha_2 \mid \cdots \mid \underline{A}\alpha_n \mid \beta_1 \mid \cdots \mid \beta_m \longrightarrow \begin{cases} A \rightarrow \beta_1 A' \mid \cdots \mid \beta_m A' \\ A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_n A' \mid \varepsilon \end{cases}$$

Après élimination de la récursivité

$$\begin{cases} E \rightarrow \overset{\alpha}{E} + T \mid \overset{\beta}{T} \\ \overline{T} \rightarrow \overline{T} * F \mid F \\ \overline{F} \rightarrow \overline{(E)} \mid id \end{cases} \longrightarrow \begin{cases} E \rightarrow TE' \\ E' \rightarrow \overline{T}E' \\ E' \rightarrow \overline{F}E' \mid \varepsilon \\ T \rightarrow F\overline{T} \\ T' \rightarrow F\overline{T}' \mid F\varepsilon \\ T' \rightarrow *F\overline{T}' \mid \varepsilon \\ F \rightarrow (\overline{E}) \mid id \\ F \rightarrow (\overline{E}) \mid id \end{cases}$$

SUPPRESSION DE LA RÉCURSIVITÉ À GAUCHE CACHÉE

RÈGLES POUR L'ÉLIMINATION DE LA RÉCURSIVITÉ À GAUCHE IMMÉDIATE

- Grouper les *A-productions* comme suit :

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n \quad \text{Aucun } \beta_i \text{ ne commence par un } A$$

- On remplace les *A-productions* par :

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \varepsilon$$

- Cet algorithme élimine toutes les récursivités à gauche immédiates.
- Il n'élimine pas les récursivités à gauche impliquant des dérivations contenant au moins deux étapes.

$$S \rightarrow Aa \mid B$$

$$A \rightarrow Ac \mid Sd \mid \varepsilon$$

Le non terminal S est récursif à gauche (mais pas immédiatement) car

$$S \Rightarrow Aa \Rightarrow Sda$$

SUPPRESSION DE LA RÉCURSIVITÉ À GAUCHE CACHÉE

ALGORITHME D'ÉLIMINATION DE LA RÉCURSIVITÉ À GAUCHE

DONNÉES : Une grammaire G sans cycles et sans productions vides

RÉSULTAT : Une grammaire équivalente à G sans récursivité à gauche

BEGIN

Ordonner les non terminaux $A_1, A_2, \dots, A_n$;

FOR $i \leftarrow 1$ **TO** n **DO**

FOR $j \leftarrow 1$ **TO** $i - 1$ **DO**

 remplacer chaque production de la forme $A_i \rightarrow A_j \gamma$ par les productions

$A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$ ($A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ sont des productions courantes)

END FOR

éliminer les récursivités immédiates des A_i -productions

END FOR

END

SUPPRESSION DE LA RÉCURSIVITÉ À GAUCHE CACHÉE

EXEMPLE :

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Ac \mid Sd \mid \varepsilon$$

- On classe les non terminaux dans l'ordre S, A
- Il n'y a pas de récursivité à gauche immédiate dans les S -productions. Rien ne se produit durant l'exécution de l'étape 2 pour $i = 1$.
- Pour $i = 2$, on substitue les S -productions dans $A \rightarrow Sd$ pour obtenir les A -productions suivantes :

$$A \rightarrow Ac \mid Aad \mid bd \mid \varepsilon$$

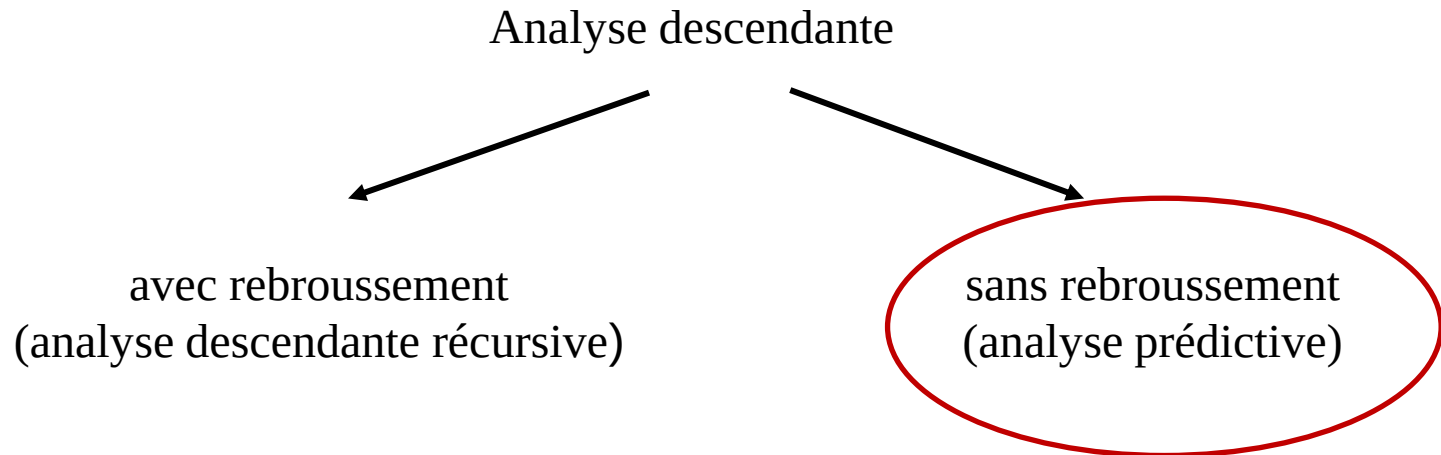
- En éliminant les récursivités à gauche immédiates, on obtient la grammaire suivante :

$$S \rightarrow Aa \mid b$$

$$A \rightarrow bdA' \mid A'$$

$$A' \rightarrow cA' \mid adA' \mid \varepsilon$$

ANALYSE SYNTAXIQUE DESCENDANTE



1. Non ambiguë
2. Factorisée à gauche
3. Non récursive à gauche

ANALYSE SYNTAXIQUE DESCENDANTE

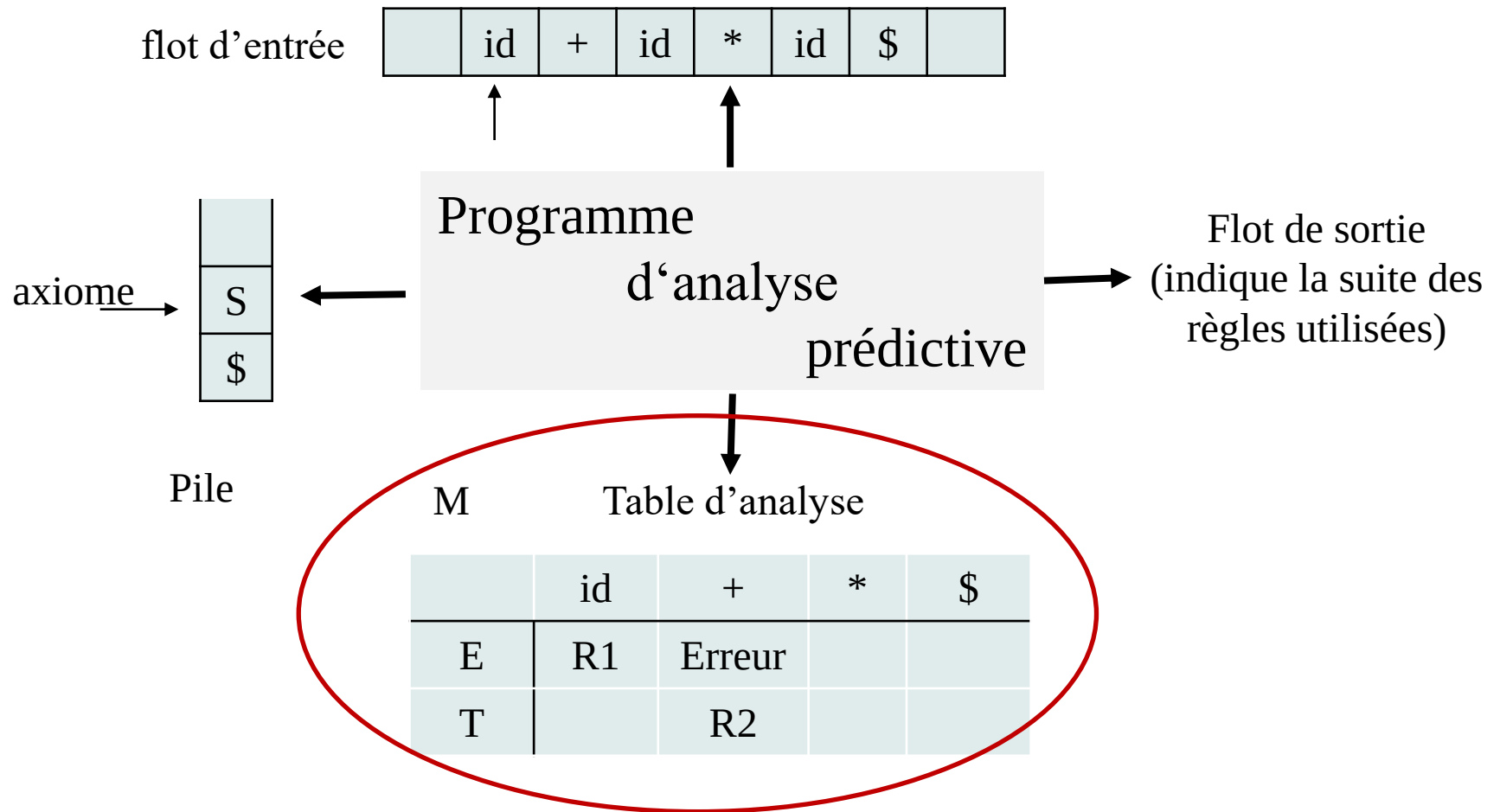
ANALYSEURS PRÉDICTIFS

- Étant donné un symbole a en entrée et un non terminal A à développer, déterminer si une alternative de la production $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$ est l'unique alternative pouvant dériver en une chaîne commençant par a .

```
instruction  →  if expression then instruction else instruction  
              |  while expression do instruction  
              |  begin instructions_list end
```

Dans ce cas de figure, les mots clés **if**, **while** et **begin** nous prédisent quelle alternative est à prendre pour développer l'instruction.

ANALYSE PRÉDICTIVE



CALCUL DU PREMIER

Soit la grammaire G suivante :

$$S \rightarrow aA \mid bB$$

$$A \rightarrow Bc \mid BC$$

$$B \rightarrow bc \mid a$$

$$C \rightarrow c \mid \varepsilon$$

$$W = a \dots ?$$

$$W = b \dots ?$$

$$W = c \dots ?$$

$$S \rightarrow^* c \dots$$

Le premier ?

$$\mathbf{Pr(S)} = \{a, b\}$$

$S \rightarrow^*$ le premier terminal ...

$$\mathbf{Pr(A)} = \mathbf{Pr(B)} = \{b, a\}$$

$$S \rightarrow aA \rightarrow aBc \dots$$

$$\mathbf{Pr(B)} = \{b, a\}$$

$$\mathbf{Pr(C)} = \{c, \varepsilon\}$$

CALCUL DU PREMIER

Soit la grammaire G suivante :

$$\begin{aligned} S &\rightarrow aA \mid bB \\ A &\rightarrow Bc \mid BCe \\ B &\rightarrow bc \mid a \mid \varepsilon \\ C &\rightarrow c \mid \varepsilon \end{aligned}$$

$$\text{Pr}(S) = \{a, b\}$$

$$A \rightarrow BC \rightarrow \varepsilon C \rightarrow \varepsilon$$

$$\text{Pr}(A) = \text{Pr}(B) = \{b, a\} \quad \text{Pr}(A) = \text{Pr}(B) \setminus \{\varepsilon\} \cup \{c\} \cup \text{Pr}(C) = \{b, a, c, e\}$$

$$\text{Pr}(B) = \{b, a\} \quad \text{Pr}(B) = \{b, a, \varepsilon\}$$

$$\text{Pr}(C) = \{c, \varepsilon\}$$

EXEMPLE D'APPLICATION

Soit la grammaire G_1 suivante :

$$\begin{array}{ll} S \rightarrow aSb \mid cd \mid bSAe & \Pr(S) = \{a, c, b\} \\ A \rightarrow aAdB \mid \varepsilon & \Pr(A) = \{a, \varepsilon\} \\ B \rightarrow bb & \Pr(B) = \{b\} \end{array}$$

Soit la grammaire G_2 suivante :

$$\begin{array}{ll} S \rightarrow ABCe & \Pr(S) = \Pr(A) \cup \Pr(B) \cup \Pr(C) \cup \{e\} = \{a, \varepsilon\} \cup \{b, c, \varepsilon\} \cup \{d, \varepsilon\} \cup \{e\} = \{a, b, c, d, e, \varepsilon\} \\ A \rightarrow aA \mid \varepsilon & \Pr(A) = \{a, \varepsilon\} \\ B \rightarrow bB \mid cB \mid \varepsilon & \Pr(B) = \{b, c, \varepsilon\} \\ C \rightarrow de \mid da \mid dA \mid \varepsilon & \Pr(C) = \{d, \varepsilon\} \end{array}$$

CALCUL DU SUIVANT

Soit la grammaire G suivante :

S	$\rightarrow$	$Aa \mid BSb$
A	$\rightarrow$	$\underline{Bc} \mid \underline{BC} \emptyset$
B	$\rightarrow$	$bc \mid a$
C	$\rightarrow$	$c \mid \varepsilon$

Le suivant ?

$S \rightarrow^* \dots A \textcircled{a} \dots$

Le premier

$S \rightarrow Aa \rightarrow B \textcircled{c} a \dots$

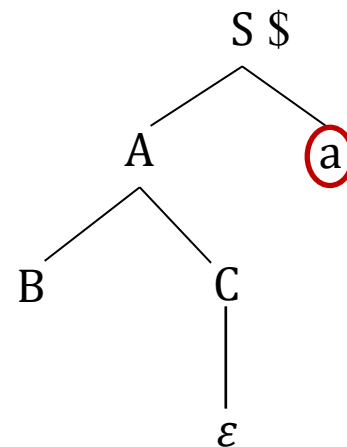
$S \rightarrow Aa \rightarrow BC \textcircled{a} \dots$

$$\text{Sui}(S) = \{\emptyset, \$\}$$

$$\text{Sui}(A) = \{a\}$$

$$\text{Sui}(B) = \text{Pr}(S) \cup \{c\} \cup \text{Pr}(C) \setminus \{\varepsilon\} \cup \text{Sui}(A)$$

$$\text{Sui}(C) = \text{Sui}(A) = \{a\}$$



EXEMPLE D'APPLICATION

Soit la grammaire G_1 suivante :

$$\begin{array}{lll}
 S \rightarrow \underline{aSb} \mid cd \mid \underline{bSAe} & \Pr(S) = \{a, c, b\} & \\
 A \rightarrow aAdB \mid \varepsilon & \Pr(A) = \{a, \varepsilon\} & Sui(A) = \{e, d\} \\
 B \rightarrow bb & \Pr(B) = \{b\} & Sui(B) = Sui(A) = \{e, d\} \\
 & & Sui(S) = \{\$, b\} \cup \Pr(A) \setminus \{\varepsilon\} \cup \{e\} = \{\$, b, a, e\}
 \end{array}$$

Soit la grammaire G_2 suivante :

$$\begin{array}{ll}
 S \rightarrow ABCe & \Pr(S) = \Pr(A) \setminus \{\varepsilon\} \cup \Pr(B) \setminus \{\varepsilon\} \cup \Pr(C) = \{a, b, c, d\} \\
 A \rightarrow aA \mid \varepsilon & \Pr(A) = \{a, \varepsilon\} \\
 B \rightarrow bB \mid cB \mid \varepsilon & \Pr(B) = \{b, c, \varepsilon\} \\
 C \rightarrow de \mid da \mid dA & \Pr(C) = \{d\} \\
 & Sui(S) = \{\$ \} \\
 & Sui(A) = \Pr(B) \setminus \{\varepsilon\} \cup \Pr(C) \cup Sui(A) \cup Sui(C) = \{b, c, d, e\} \\
 & Sui(B) = \Pr(C) \cup Sui(B) = \{d\} \\
 & Sui(C) = \{e\}
 \end{array}$$

EXEMPLE D'APPLICATION

Soit la grammaire G suivante :

$$\begin{cases} E \rightarrow \underline{TE'} \\ E' \rightarrow \underline{+TE'} \mid \varepsilon \\ T \rightarrow \underline{FT'} \\ T' \rightarrow \underline{*FT'} \mid \varepsilon \\ F \rightarrow \underline{(E)} \mid id \end{cases}$$

$$Sui(E) = \{ \$,) \}$$

$$Sui(E') = Sui(E) \cup Sui(E') = \{ \$,) \}$$

$$Sui(T) = Pr(E') \setminus \{ \varepsilon \} \cup Sui(E) \cup Sui(E') = \{ + , \$,) \}$$

$$Sui(T') = Sui(T) \cup Sui(T') = \{ + , \$,) \}$$

$$Sui(F) = Pr(T') \setminus \{ \varepsilon \} \cup Sui(T) \cup Sui(T') = \{ * , + , \$,) \}$$

$$Pr(E) = Pr(T) = Pr(F) = \{ (, id \}$$

$$Pr(E') = \{ + , \varepsilon \}$$

$$Pr(T) = Pr(F) = \{ (, id \}$$

$$Pr(T') = \{ * , \varepsilon \}$$

$$Pr(F) = \{ (, id \}$$

TABLE D'ANALYSE

Soit la grammaire G suivante :

$$\begin{cases} E \rightarrow TE' \\ E' \rightarrow +TE' \mid \varepsilon \\ T \rightarrow FT' \\ T' \rightarrow *FT' \mid \varepsilon \\ F \rightarrow (E) \mid id \end{cases}$$

$$Sui(E) = \{ \$,) \}$$

$$\underline{Sui(E') = Sui(E) \cup Sui(E') = \{ \$,) \}}$$

$$Sui(T) = Pr(E') \setminus \{ \varepsilon \} \cup Sui(E) \cup Sui(E') = \{ + , \$,) \}$$

$$\underline{Sui(T') = Sui(T) \cup Sui(T') = \{ + , \$,) \}}$$

$$Sui(F) = Pr(T') \setminus \{ \varepsilon \} \cup Sui(T) \cup Sui(T') = \{ * , + , \$,) \}$$

$$Pr(E) = Pr(T) = Pr(F) = \{ (, id \}$$

$$Pr(E') = \{ +, \varepsilon \}$$

$$Pr(T) = Pr(F) = \{ (, id \}$$

$$Pr(T') = \{ *, \varepsilon \}$$

$$Pr(F) = \{ (, id \}$$

	$+$	$*$	$($	$)$	id	$\$$
E	<i>Erreur</i>	<i>Erreur</i>	$E \rightarrow TE'$	<i>Erreur</i>	$E \rightarrow TE'$	<i>Erreur</i>
E'	$E' \rightarrow +TE'$	<i>Erreur</i>	<i>Erreur</i>	$E' \rightarrow \varepsilon$	<i>Erreur</i>	$E' \rightarrow \varepsilon$
T	<i>Erreur</i>	<i>Erreur</i>	$T \rightarrow FT'$	<i>Erreur</i>	$T \rightarrow FT'$	<i>Erreur</i>
T'	$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$	<i>Erreur</i>	$T' \rightarrow \varepsilon$	<i>Erreur</i>	$T' \rightarrow \varepsilon$
F	<i>Erreur</i>	<i>Erreur</i>	$F \rightarrow (E)$	<i>Erreur</i>	$F \rightarrow id$	<i>Erreur</i>

TABLE D'ANALYSE

$T' \rightarrow \varepsilon ?$

Soit $W = id + id ?$

$E \rightarrow TE' \rightarrow FT'E' \rightarrow idT'E' \rightarrow idE' \rightarrow id + TE'$

Soit la grammaire G suivante :

$$\begin{cases} E \rightarrow TE' \\ E' \rightarrow +TE' \mid \varepsilon \\ T \rightarrow FT' \\ T' \rightarrow *FT' \mid \varepsilon \\ F \rightarrow (E) \mid id \end{cases}$$

$$Sui(E) = \{ \$,) \}$$

$$Sui(E') = Sui(E) \cup Sui(E') = \{ \$,) \}$$

$$Sui(T) = Pr(E') \setminus \{ \varepsilon \} \cup Sui(E) \cup Sui(E') = \{ + , \$,) \}$$

$$\underline{Sui(T') = Sui(T) \cup Sui(T') = \{ + , \$,) \}}$$

$$Sui(F) = Pr(T') \setminus \{ \varepsilon \} \cup Sui(T) \cup Sui(T') = \{ * , + , \$,) \}$$

$$Pr(E) = Pr(T) = Pr(F) = \{ (, id \}$$

$$Pr(E') = \{ + , \varepsilon \}$$

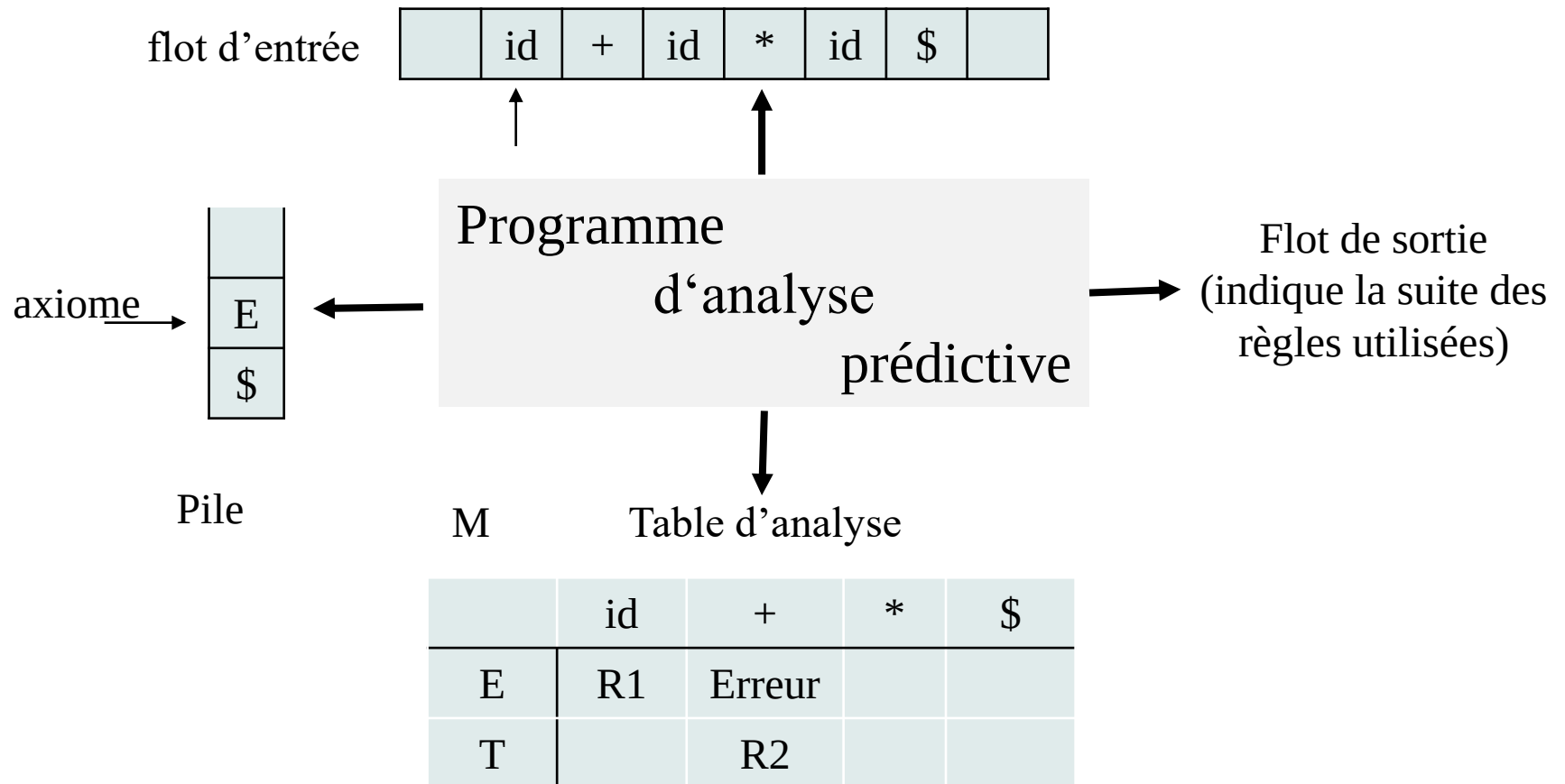
$$Pr(T) = Pr(F) = \{ (, id \}$$

$$\underline{Pr(T') = \{ * , \varepsilon \}}$$

$$Pr(F) = \{ (, id \}$$

	$+$	$*$	$($	$)$	id	$\$$
E	<i>Erreur</i>	<i>Erreur</i>	$E \rightarrow TE'$	<i>Erreur</i>	$E \rightarrow TE'$	<i>Erreur</i>
E'	$E' \rightarrow +TE'$	<i>Erreur</i>	<i>Erreur</i>	$E' \rightarrow \varepsilon$	<i>Erreur</i>	$E' \rightarrow \varepsilon$
T	<i>Erreur</i>	<i>Erreur</i>	$T \rightarrow FT'$	<i>Erreur</i>	$T \rightarrow FT'$	<i>Erreur</i>
T'	$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$	<i>Erreur</i>	$T' \rightarrow \varepsilon$	<i>Erreur</i>	$T' \rightarrow \varepsilon$
F	<i>Erreur</i>	<i>Erreur</i>	$F \rightarrow (E)$	<i>Erreur</i>	$F \rightarrow id$	<i>Erreur</i>

ANALYSE PRÉDICTIVE



ANALYSE SYNTAXIQUE DESCENDANTE

ANALYSE PRÉDICTIVE NON RÉCURSIVE

- Tableau d'analyse à deux dimensions $M[A, a]$, A est non-terminal, a est terminal ou \$.
- Initialement la pile contient l'axiome de la grammaire au dessus du symbole \$.
- X est le symbole au sommet de la pile et a le symbole d'entrée courant déterminent l'action de l'analyseur. Trois possibilités :
 - Si $X = a = \$$, Arrêt de l'analyseur et fin réussi de l'analyse.
 - Si $X = a \neq \$$, l'analyseur enlève X de la pile et avance son pointeur vers le symbole suivant.
 - Si X est un non-terminal, le programme consulte l'entrée $M[X, a]$ de la table d'analyse M qui est soit une X -production soit une erreur.
 - Si X -production, remplacer X au sommet de la pile par la X -production (*).
 - Si erreur, appeler la procédure de récupération sur erreur.

(*) $M[X, a] = \{X \rightarrow UVW\}$, remplacer X au sommet de la pile par WVU (avec U au sommet)

ANALYSE SYNTAXIQUE DESCENDANTE

ALGORITHME D'ANALYSE PRÉDICTIVE NON RÉCURSIVE

DONNÉES : Une chaîne w et une table d'analyse M pour une grammaire G

RÉSULTAT : Si w est dans $\mathcal{L}(G)$, une dérivation gauche pour w , erreur sinon.

Positionner le pointeur source ps sur le premier symbole de $w\$$;

```
WHILE  $X \neq \$$  DO
  soit  $X$  le symbole en sommet de la pile et  $a$  le symbole pointé par  $ps$ :
  IF  $X$  est un terminal THEN
    IF  $X = a$  THEN
      enlever  $X$  de la pile et avancer  $ps$ ;
    ELSE
      error ();
    END IF
  ELSE /*  $X$  est un non-terminal */
    IF  $M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$  THEN
      placer  $Y_k, Y_{k-1}, \dots, Y_1$  sur la pile avec  $Y_1$  au sommet;
      émettre en sortie la production  $X \rightarrow Y_1 Y_2 \dots Y_k$ 
    ELSE
      error ();
    END IF
  END IF
END WHILE
```

ANALYSE DESCENDANTE

Soit $W = id + (id * id)\$$?

	$+$	$*$	$($	$)$	id	$\$$
E	Erreur	Erreur	$E \rightarrow TE'$	Erreur	$E \rightarrow TE'$	Erreur
E'	$E' \rightarrow +TE'$	Erreur	Erreur	$E' \rightarrow \varepsilon$	Erreur	$E' \rightarrow \varepsilon$
T	Erreur	Erreur	$T \rightarrow FT'$	Erreur	$T \rightarrow FT'$	Erreur
T'	$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$	Erreur	$T' \rightarrow \varepsilon$	Erreur	$T' \rightarrow \varepsilon$
F	Erreur	Erreur	$F \rightarrow (E)$	Erreur	$F \rightarrow id$	Erreur

Symbole de fin de chaîne

Sommet de la pile

Rq : On empile à l'envers

Pile	Entrée	Sortie
$\$E$	$id + (id * id)\$$	$E \rightarrow TE'$
$\$E'T$	$id + (id * id)\$$	$T \rightarrow FT'$
$\$E'T'F$	$id + (id * id)\$$	$F \rightarrow id$
$\$E'T'id$	$id + (id * id)\$$	
$\$E'T'$	$+(id * id)\$$	$T' \rightarrow \varepsilon$
$\$E'$	$+(id * id)\$$	

ANALYSE DESCENDANTE

Soit $W = id + (id * id)\$$?

	$+$	$*$	$($	$)$	id	$\$$
E	<i>Erreur</i>	<i>Erreur</i>	$E \rightarrow TE'$	<i>Erreur</i>	$E \rightarrow TE'$	<i>Erreur</i>
E'	$E' \rightarrow +TE'$	<i>Erreur</i>	<i>Erreur</i>	$E' \rightarrow \varepsilon$	<i>Erreur</i>	$E' \rightarrow \varepsilon$
T	<i>Erreur</i>	<i>Erreur</i>	$T \rightarrow FT'$	<i>Erreur</i>	$T \rightarrow FT'$	<i>Erreur</i>
T'	$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$	<i>Erreur</i>	$T' \rightarrow \varepsilon$	<i>Erreur</i>	$T' \rightarrow \varepsilon$
F	<i>Erreur</i>	<i>Erreur</i>	$F \rightarrow (E)$	<i>Erreur</i>	$F \rightarrow id$	<i>Erreur</i>

Pile	Entrée	Sortie
$\$E$	$id + (id * id)\$$	$E \rightarrow TE'$
$\$E'T$	$id + (id * id)\$$	$T \rightarrow FT'$
$\$E'T'F$	$id + (id * id)\$$	$F \rightarrow id$
$\$E'T'id$	id $ + (id * id)\$$	
$\$E'T'$	$+(id * id)\$$	$T' \rightarrow \varepsilon$
$\$E'$	$+(id * id)\$$	$E' \rightarrow +TE'$
$\$E'T+$	$+(id * id)\$$	
$\$E'T$	$(id * id)\$$	$T \rightarrow FT'$
$\$E'T'F$	$(id * id)\$$	$F \rightarrow (E)$
$\$E'T')E($	$(id * id)\$$	
$\$E'T')E($	$id + (id * id)\$$	$E \rightarrow TE'$

ANALYSE

Soit $W = id +$

	$+$	$*$	$($	$)$	id	$\$$
E	<i>Erreur</i>	<i>Erreur</i>	$E \rightarrow TE'$	<i>Erreur</i>	$E \rightarrow TE'$	<i>Erreur</i>

Pile	Entrée	Sortie	<i>Erreur</i>	$E' \rightarrow \varepsilon$
$\$E$	$id + (id * id)\$$	$E \rightarrow TE'$	FT'	<i>Erreur</i>
$\$E'T$	$id + (id * id)\$$	$T \rightarrow FT'$	<i>Erreur</i>	$T' \rightarrow \varepsilon$
$\$E'T'F$	$id + (id * id)\$$	$F \rightarrow id$	$\rightarrow id$	<i>Erreur</i>
$\$E'T'id$	$id + (id * id)\$$			
$\$E'T'$	$+(id * id)\$$	$T' \rightarrow \varepsilon$		
$\$E'$	$+(id * id)\$$	$E' \rightarrow +TE'$		
$\$E'T+$	$+(id * id)\$$			
$\$E'T$	$(id * id)\$$	$T \rightarrow FT'$		
$\$E'T'F$	$(id * id)\$$	$F \rightarrow (E)$		
$\$E'T')E\{$	$\{(id * id)\$$			
$\$E'T')E$	$id * id)\$$	$E \rightarrow TE'$		
$\$E'T')E'T$	$id * id)\$$	$T \rightarrow FT'$		
$\$E'T')E'T'F$	$id * id)\$$	$F \rightarrow id$		
$\$E'T')E'T'id$	$id * id)\$$			
$\$E'T')E'T'$	$* id)\$$	$T' \rightarrow * FT'$		

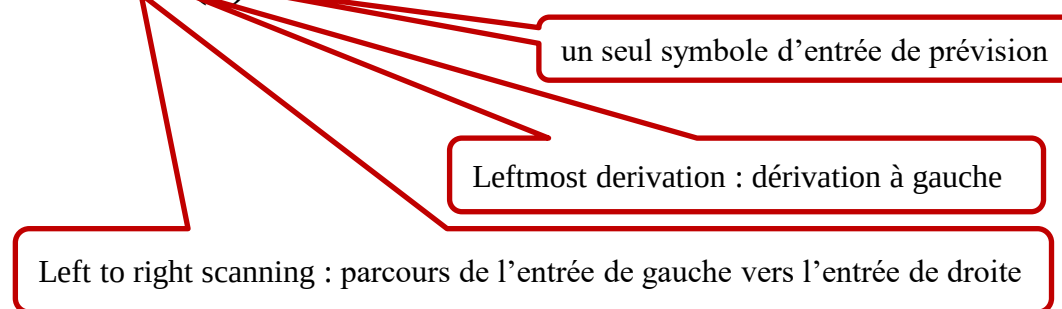
ANALYSE

Soit $W = id +$

$\$E'T$	$id + (id * id)\$$	$T \rightarrow FT'$	d	\$
$\$E'T'F$	$id + (id * id)\$$	$F \rightarrow id$		
$\$E'T' \cancel{id}$	$\cancel{id} + (id * id)\$$			<i>Erreur</i>
$\$E'T'$	$+(id * id)\$$	$T' \rightarrow \varepsilon$		<i>Erreur</i>
$\$E'$	$+(id * id)\$$	$E' \rightarrow +TE'$	<i>Erreur</i>	<i>Erreur</i>
$\$E'T - +$	$\cancel{+}(id * id)\$$		<i>Erreur</i>	$T' \rightarrow \varepsilon$
$\$E'T$	$(id * id)\$$	$T \rightarrow FT'$	$\rightarrow id$	<i>Erreur</i>
$\$E'T'F$	$(id * id)\$$	$F \rightarrow (E)$		
$\$E'T')E \{$	$\{(id * id)\$$			
$\$E'T')E$	$id * id)\$$	$E \rightarrow TE'$		
$\$E'T')E'T$	$id * id)\$$	$T \rightarrow FT'$		
$\$E'T')E'T'F$	$id * id)\$$	$F \rightarrow id$		
$\$E'T')E'T' \cancel{id}$	$\cancel{id} * id)\$$			
$\$E'T')E'T'$	$* id)\$$	$T' \rightarrow * FT'$		
$\$E'T')E'T'F - *$	$\cancel{*} id)\$$			
$\$E'T')E'T'F$	$id)\$$	$F \rightarrow id$		
$\$E'T')E'T' \cancel{id}$	$\cancel{id})\$$			
$\$E'T')E'T'$	$)\$$	$T' \rightarrow \varepsilon$		
$\$E'T')E'$	$)\$$	$E' \rightarrow \varepsilon$		

GRAMMAIRE LL(1)

- Pour certaines grammaires, la table d'analyse peut avoir des entrées définies de façon multiple.
- La grammaire dont la table d'analyse n'a aucune entrée définie de façon multiple est dite LL(1).



Une grammaire est LL(1) ➔

- Factorisée à gauche
- Non réursive à gauche
- Non ambiguë